

Micro-processeur

Pierre Bodet, Charles Dai, Max Holemans, Joachim Marchal

ENS - PSL

27/01/2026

- 1 ALU
- 2 Assembleur
- 3 Processeur
- 4 Horloge
- 5 Opérations entières additionnelles
- 6 Floats - Format 16 bits
- 7 Floats - Format 32 bits

- 1 ALU
- 2 Assembleur
- 3 Processeur
- 4 Horloge
- 5 Opérations entières additionnelles
- 6 Floats - Format 16 bits
- 7 Floats - Format 32 bits

Au commencement était l'ALU



ALU - Coeur

n-bit adder en carry-lookahead

Explications

cf TD2

ALU - opérations

Opérations

Arithmétique : add, sub, mul

Logique : not, and, or, xor

Shifts : sll, srl (repris du code de Pierre)

Flags

C : Carry

V : Overflow

N : Negative

Z : Zero

- ① ALU
- ② Assembleur
- ③ Processeur
- ④ Horloge
- ⑤ Opérations entières additionnelles
- ⑥ Floats - Format 16 bits
- ⑦ Floats - Format 32 bits

Diverse opérations

<i>add</i>	<i>rd, rs1, rs2</i>
------------	---------------------

<i>addi</i>	<i>rd, rs1, imm</i>
-------------	---------------------

<i>and</i>	<i>rd, rs1, rs2</i>
------------	---------------------

<i>andi</i>	<i>rd, rs1, imm</i>
-------------	---------------------

<i>or</i>	<i>rd, rs1, rs2</i>
-----------	---------------------



<i>ori</i>	<i>rd, rs1, imm</i>
------------	---------------------

Défis de la conversion

addi

add immediate

31-27	26-25	24-20	19-15	14-12	11-7	6-2	1-0
imm[11:0]			rs1	000	rd	00100	11

Sauts

```
correct_y:
    blt x31 x23 correct_d
    sub x31 x31 x22
    addi x6 x6 1
    jal x0 correct_y
correct_d:
    blt x31 x22 correct_h
    sub x31 x31 x22
    addi x5 x5 1
    jal x0 correct_d
correct_h:
    blt x31 x21 correct_m
    sub x31 x31 x21
```

aaaAAAAAAhhh

jal

31-27	26-25	24-20	19-15	14-12	11-7	6-2	1-0
offset[20 10:1 11			19:12]		rd	11011	11



- ① ALU
- ② Assembleur
- ③ Processeur
- ④ Horloge
- ⑤ Opérations entières additionnelles
- ⑥ Floats - Format 16 bits
- ⑦ Floats - Format 32 bits

Interaction avec le compilateur

```

signal_lookup_table = [
    "000" + "00000000" + "0100000", # add
    "000" + "00000000" + "1100000", # addi
    "000" + "00100000" + "0100000", # and
    "000" + "00100000" + "1100000", # andi
    "000" + "00010000" + "0100000", # or
    "000" + "00010000" + "1100000", # ori
    "000" + "01000000" + "0100000", # xor
    "000" + "01000000" + "1100000", # xori
    "000" + "10000000" + "0100000", # sub
    "000" + "00000000" + "0010000", # sw
    "000" + "00000010" + "0100000", # sll
    "000" + "00000010" + "1100000", # slli
    "000" + "00000100" + "0100000", # srl
    "000" + "00000100" + "1100000", # srli
    "000" + "00000001" + "0100000", # mul
    "000" + "00000000" + "0100100", # lw
    "000" + "00000000" + "0101000", # jal
    "000" + "10000000" + "0000010", # beq
    "000" + "10000000" + "0000010", # bne
    "000" + "10000000" + "0000010", # blt
    "000" + "10000000" + "0000010", # bge
    "000" + "00000000" + "0100001", # rdtime
    "100" + "00000000" + "0000000", # fadd
    "100" + "00000000" + "0000000", # fsub
    "100" + "00000000" + "0000000", # fmul
    "110" + "00000000" + "0000000", # fdiv
    "110" + "00000000" + "0000000", # ffisqrt
    "100" + "00000000" + "0000010", # feq
    "001" + "00000000" + "0000000", # fcvt.w
    "001" + "00000000" + "0000000", # fcvt.s
    "000" + "00000000" + "0100001", # rdclock
]

```

Interaction avec la netlist et l'horloge

Le processeur prend en entrée un front montant de clock et le nombre de secondes écoulées depuis le 1er janvier 2026, qui sont automatiquement remplies par le programme de netlist.

Le programme d'horloge peut alors lire ces deux signaux à l'aide d'instructions rdclock et rdtime.

Interaction avec l'ALU et les flottants

Max et Pierre ont codé leurs netlists sous forme de modules carotte, qui sont importées par le processeur. La look-up table permet de récupérer les signaux attendus en arguments par les modules de l'ALU et des flottants.

Processeur avec ALU et multiplieur : 5319 lignes de netlist

Processeur avec ALU, multiplieur et flottants : 150752 lignes de netlist

- ① ALU
- ② Assembleur
- ③ Processeur
- ④ Horloge
- ⑤ Opérations entières additionnelles
- ⑥ Floats - Format 16 bits
- ⑦ Floats - Format 32 bits

Horloge - introduction

- Espace disponible : 256 instructions dans la ROM
- Instructions standard utilisées : add, addi, sub, jal, beq, bne, blt, slli
- Nouvelles instructions
 - rdtime : reçoit une sorte de Unix time (secondes depuis 00 :00 01/01/2026) et le place dans un registre
 - rdclock : reçoit la valeur d'une clock, qui oscille entre 0 et 1 avec une période de 1 seconde

Horloge - Registres

Différentes utilités des registres

- Contenir des constantes
- Maintenir le valeurs d'intérêt : secondes, minutes, heures...
- Recevoir et manipuler les temps d'entrée

Horloge - Fonctionnement

Fonctionnement du programme

- On part de l'état 00 :00 01/01/2026
- On récupère le temps actuel et on le rejoint en accéléré
- On boucle sur la clock, avec un incrémentation simple des valeurs lors des fronts montants

Fonctionnement en fast-forward

On ne fait pas d'appel au temps (rdtime ou rdclock), on par de 00 :00 01/01/2026 et on incrémente seconde par seconde en boucle.

Horloge - Particularités

Calcul du mois :

```
1      jan:
2          addi x11 x5 -1
3          addi x8 x0 1
4          blt x11 x24 ret_month
5          sub x11 x11 x24
6      feb:
7          addi x8 x8 1
8          add x12 x26 x7
9          blt x11 x12 ret_month
10         sub x11 x11 x12
11      mar:
12         addi x8 x8 1
13         blt x11 x24 ret_month
14         sub x11 x11 x24
```

Horloge - Particularités

- Année bissextile (dans une certaine mesure)

"D'autres... Oui ! Mais moi, j'aime ceux qui vivent
aujourd'hui sur la même terre que moi, et c'est eux que je
salue."

Kaliayev - Les Justes, Camus

- Représentation des jours

- ① ALU
- ② Assembleur
- ③ Processeur
- ④ Horloge
- ⑤ Opérations entières additionnelles
- ⑥ Floats - Format 16 bits
- ⑦ Floats - Format 32 bits

Décalages binaires (Bit Shift)

Principe

Décaler les bits d'un nombre vers la gauche ou la droite

- Décalage à droite : ajoute des zéros à droite (multiplication par 2^n)
- Décalage à gauche : ajoute des zéros à gauche (division par 2^n)
- Implémentation progressive : décalage du nombre de bits demandé

Utilité : essentiel pour normaliser les mantisses

Comparaisons

Fonctions implémentées

- Égalité ($a = b$)
- Supérieur ou égal ($a \geq b$)
- Strictement supérieur ($a > b$)
- Comparaison bit par bit de poids fort vers poids faible
- Mémorisation du résultat strictement supérieur
- Gestion du cas d'égalité parfaite

Utilité : choix des opérandes, détection de cas spéciaux

Multiplication

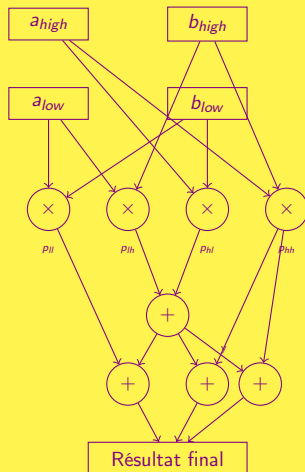
Algorithme de Karatsuba

Multiplication récursive divisant le problème en sous-problèmes

Pour n bits :

- Diviser a et b en parties haute et basse
- Calculer 4 produits partiels
- Recombiner avec additions et décalages

Complexité : $O(n^{1.58})$ au lieu de $O(n^2)$



- ① ALU
- ② Assembleur
- ③ Processeur
- ④ Horloge
- ⑤ Opérations entières additionnelles
- ⑥ Floats - Format 16 bits
- ⑦ Floats - Format 32 bits



Encodage/Décodage IEEE 754 (16 bits)

Format half-precision

Signe (1 bit)	Exposant (5 bits)	Mantisse (10 bits)
---------------	-------------------	--------------------

- Signe : bit 15 (0 = positif, 1 = négatif)
- Exposant : bits 10-14, biaisé de 127
- Mantisse : bits 0-9, bit implicite ajouté si exposant $\neq 0$
- Valeur réelle : $(-1)^s \times 1.m \times 2^{e-127}$

Addition flottante

Étapes principales

- ① Aligner les exposants (décaler la mantisse du plus petit)
 - ② Additionner ou soustraire les mantisses selon les signes
 - ③ Normaliser le résultat (supprimer les zéros à gauche)
 - ④ Ajuster l'exposant en conséquence
- Comparaison préalable pour déterminer le plus grand nombre
 - Gestion du bit de retenue pour ajuster mantisse et exposant
 - Inversion des bits pour la soustraction

Multiplication flottante

Algorithme simplifié

- 1 Multiplier les mantisses (algorithme de Karatsuba)
- 2 Additionner les exposants
- 3 Soustraire le biais (127)
- 4 Calculer le signe : XOR des signes
- 5 Normaliser le résultat

Formule : $(s_a \oplus s_b, e_a + e_b - 127, m_a \times m_b)$

Division flottante

Division des mantisses (division entière)

Algorithme itératif appris en primaire

- Soustraire le diviseur du dividende si possible
- Décaler et répéter pour obtenir chaque bit du quotient
- Exposant : $e_a - e_b + 127$
- Signe : $s_a \oplus s_b$
- Détection des NaN (division par zéro, opérandes invalides)

Conversion flottant → entier

Trois modes d'arrondi implémentés

- `round_down` : vers $-\infty$
- `round_up` : vers $+\infty$
- `round_nearest_to_even` : au pair le plus proche

Principe :

- ① Extraire signe, exposant, mantisse
- ② Décaler la mantisse selon l'exposant
- ③ Appliquer la règle d'arrondi
- ④ Gérer le signe (complément à 2 si négatif)

- ① ALU
- ② Assembleur
- ③ Processeur
- ④ Horloge
- ⑤ Opérations entières additionnelles
- ⑥ Floats - Format 16 bits
- ⑦ Floats - Format 32 bits

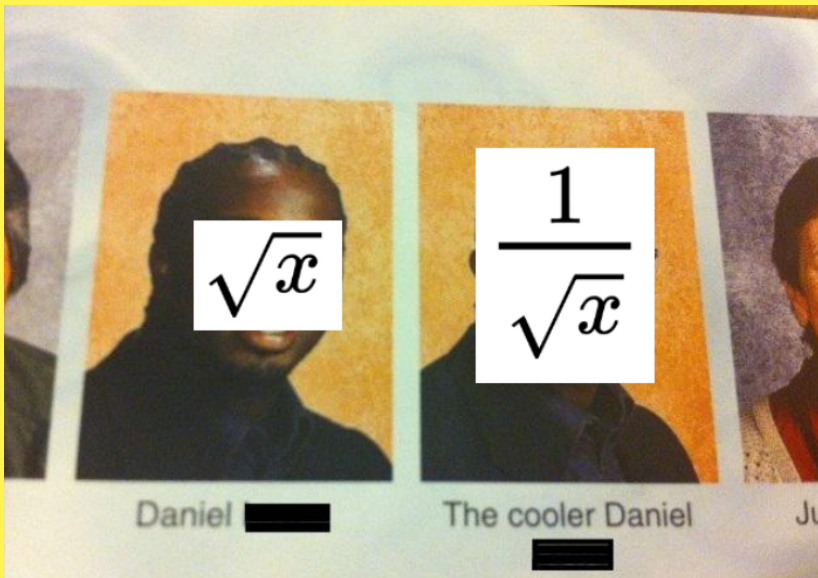
Extension 16 bits → 32 bits

Passage au format simple précision

- Format 32 bits : 1 bit signe, 8 bits exposant, 23 bits mantisse
- Nouveau biais : 127 (au lieu de 15)
- Mêmes algorithmes adaptés aux nouvelles tailles

Implémentation :

- Il faut modifier quelques fonctions seulement, puisque tous les autres codes réutilisent quelques fonctions pour être le plus général possible
- Tout est détaillé précisément dans le README.md



Fast Inverse Square Root

Algorithme de Quake III

Calcul rapide de $1/\sqrt{x}$ par approximation

Étapes :

- ① Nombre magique : `0x5f3759df`
- ② Calcul comme si c'était des entiers : $i = \text{magic} - (x \ggg 1)$
- ③ Itération de Newton : $y_{n+1} = y_n \times (1.5 - 0.5 \times x \times y_n^2)$



Fin

Merci de nous avoir écouté !

[isa,] Risc-v instruction set.

https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf.

[sit,] Version claire de l'instruction set ci-dessus.

<https://msyksphinz-self.github.io/riscv-isadoc/html/index.html>.